

Padrões para a Camada de Negócio

```
graph TD; P[Presentation] --- BL[Business Logic]; BL --- DS[Data Source]; DS --- DB[(Database layer)];
```

Presentation

Business Logic

Data Source

Database layer

Martin Fowler (software engineer)

22 languages

Article Talk

Read Edit View history Tools

From Wikipedia, the free encyclopedia

Martin Fowler (18 December 1963) is a British [software developer](#),^[2] author and international public speaker on software development, specialising in [object-oriented analysis and design](#), [UML](#), [patterns](#), and [agile software development](#) methodologies, including [extreme programming](#).

His 1999 book *Refactoring* popularised the practice of [code refactoring](#).^[3] In 2004 he introduced a new [architectural pattern](#), called Presentation Model (PM).^[4]

Biography [edit]

Fowler was born and grew up in [Walsall](#), England, where he went to [Queen Mary's Grammar School](#) for his secondary education. He graduated at [University College London](#) in 1986. In 1994, he moved to the United States, where he lives near [Boston](#), [Massachusetts](#) in the suburb of [Melrose](#).^[1]

Fowler started working with software in the early 1980s. Out of university in 1986 he started working in software development for [Coopers & Lybrand](#) until 1991.^[5] In 2000 he joined [ThoughtWorks](#), a systems integration and consulting company,^[1] where he was Chief Scientist.^[6]

Fowler has written nine books on the topic of software development. He is a member of the [Agile Alliance](#) and helped create the [Manifesto for Agile Software Development](#) in 2001, along with 16 fellow signatories.^[7] He maintains a *bliki*, a mix of [blog](#) and [wiki](#). He popularised the term [Dependency Injection](#) as a form of [Inversion of Control](#).^{[8][9]}

Publications [edit]

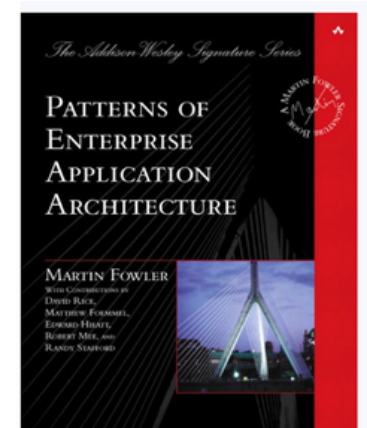
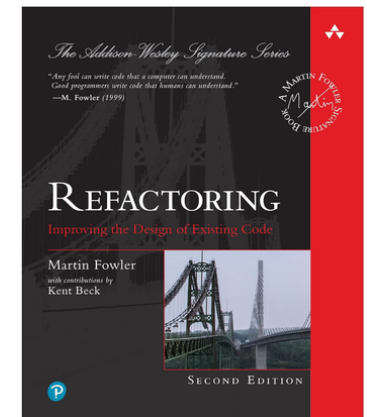
- 1996. *Analysis Patterns: Reusable Object Models*. Addison-Wesley. ISBN 0-201-89542-0.
- 1997. *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley. ISBN 978-0-201-32563-8.
- 1999. *Refactoring: Improving the Design of Existing Code*, With [Kent Beck](#), John Brant, [William Opdyke](#), and Don Roberts (June 1999). Addison-Wesley. ISBN 0-201-48567-2.
- 2000. *Planning Extreme Programming*. With [Kent Beck](#). Addison-Wesley. ISBN 0-201-71091-9.
- 2002. *Patterns of Enterprise Application Architecture*. With David Rice, Matthew Foemmel, Edward Hieatt, Robert Mee, and Randy Stafford. Addison-Wesley. ISBN 0-321-12742-0.

Martin Fowler



Fowler in 2015

Born	18 December 1963 (age 61) Walsall, England, UK
Education	University College London (BSc, 1986) ^[1]
Occupation(s)	Software developer, author, public speaker
Employer	ThoughtWorks
Website	martinfowler.com



Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model
	Table Module
	Transaction Script
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper

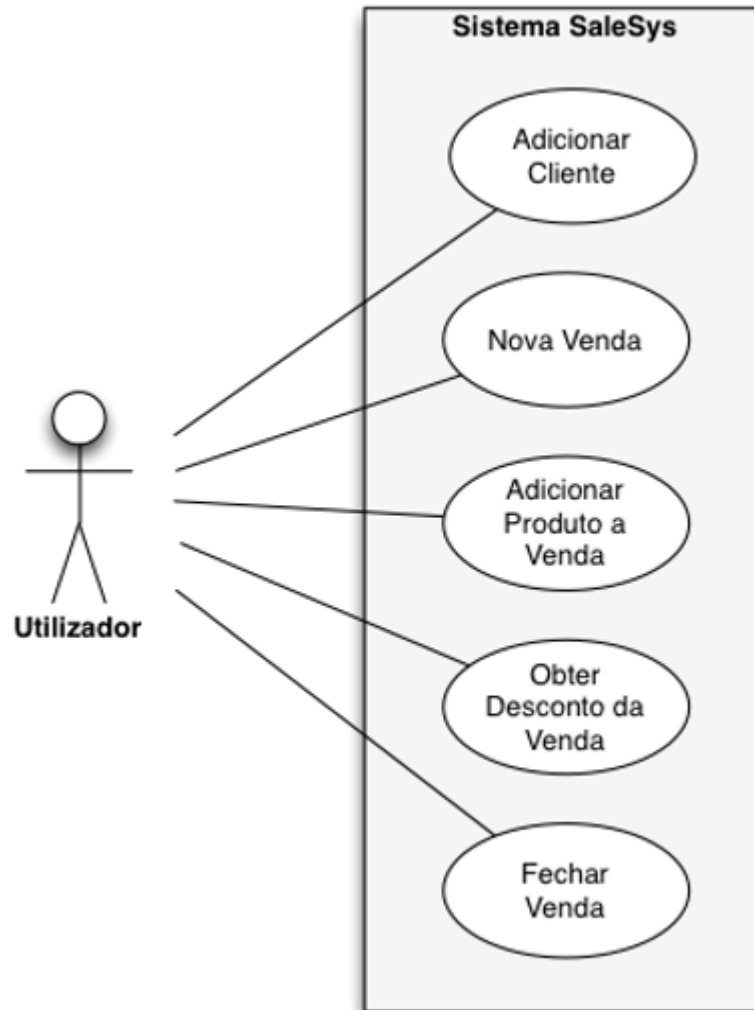
SaleSys

Aplicação Exemplo

- É um aplicação fictícia, muito simples, que é usada na disciplina ao longo do semestre para ilustrar os conceitos e as técnicas que vão sendo apresentadas
- O propósito da aplicação é fazer a gestão da carteira de clientes e de vendas de uma empresa
- Casos de uso:
 - Adicionar cliente
 - Criar venda
 - Acrescentar produtos a uma venda em aberto
 - Obter valor de desconto da venda
 - Concluir venda

SaleSys

Diagrama de Casos de Uso



Presentation

Business Logic

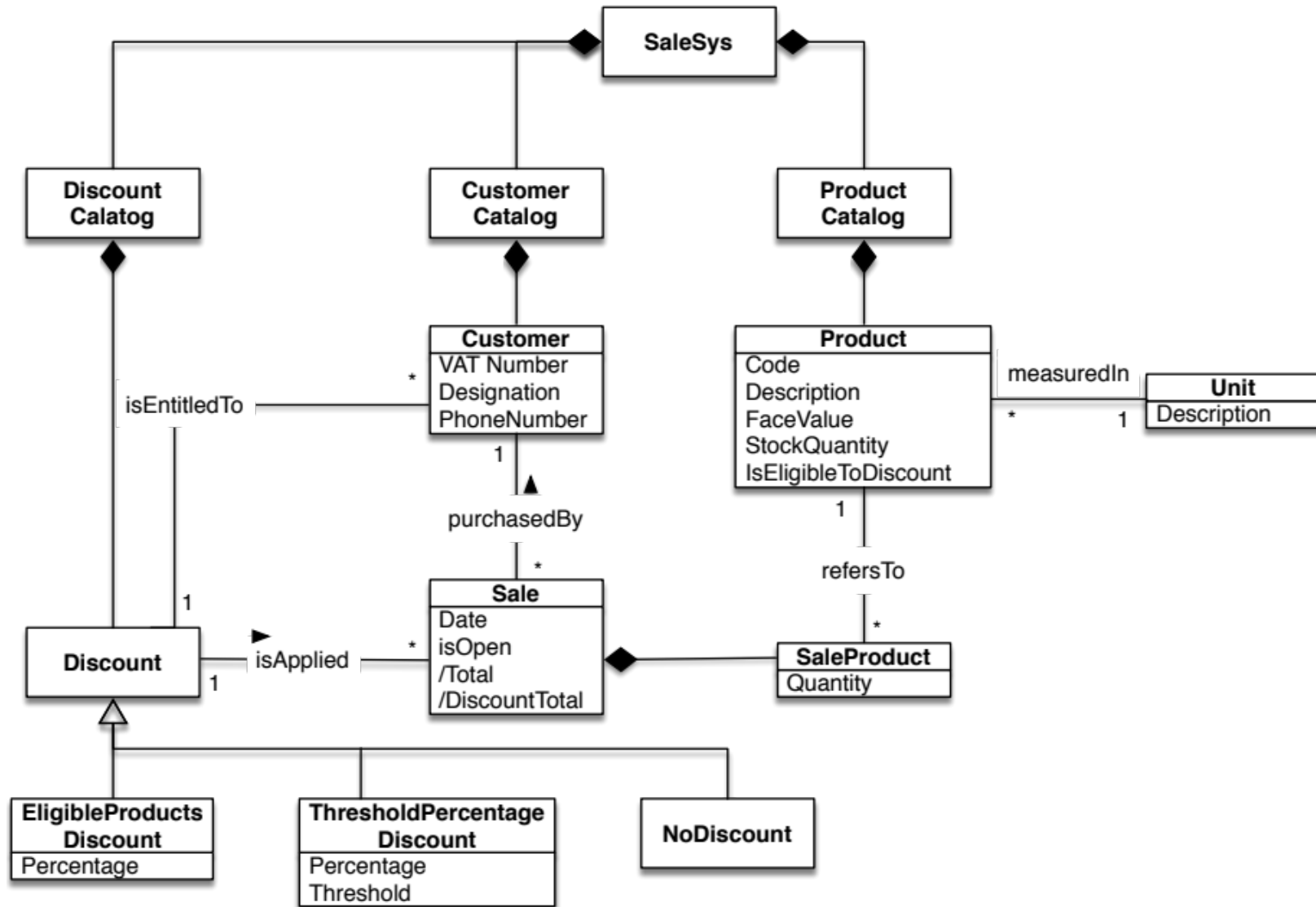
Data Source

Domain Model

- Organizar o código baseado em **conceitos do domínio**, suas **características** e **associações** (abordagem de DCO).
- Adequado para quando a lógica tem um grau de complexidade elevado.
- **Vantagens:** Forma de dominar a complexidade, dividindo o problema em objectos responsáveis pelo seu âmbito (*concern*).
- **Desvantagens:** Requer experiência em OO e esforço na modelação; A persistência em BD relacionais é um desafio.

SaleSys

Modelo de Domínio



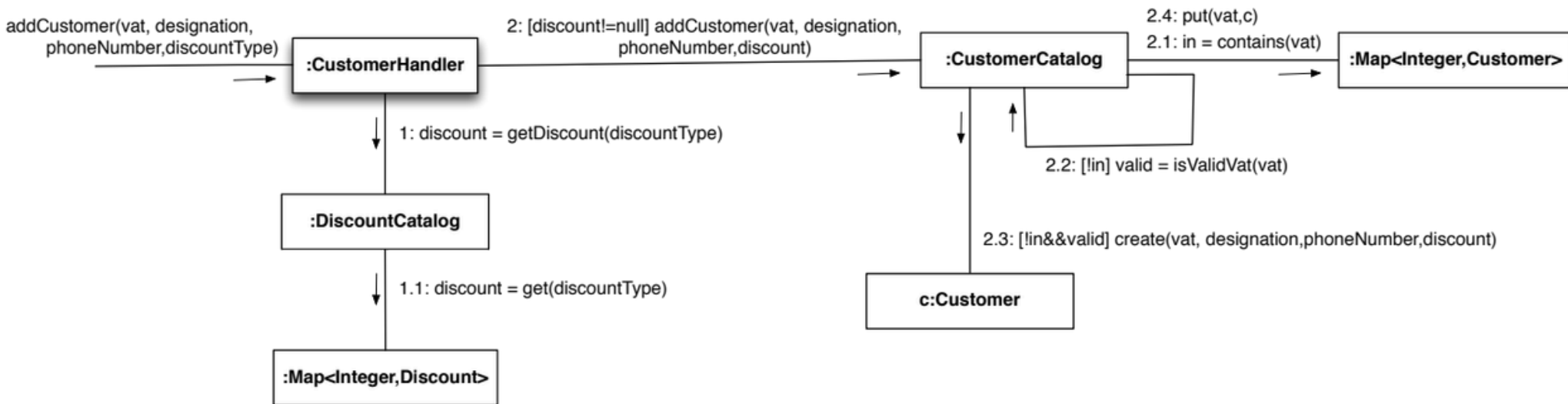
SaleSys

Caso de Uso: Adicionar Cliente

- Se o código do tipo de desconto é válido
e o VAT é válido
e o Nome e o Telefone estão preenchidos,
→ **Registrar cliente**

SaleSys

Adicionar Cliente



SaleSys

Código

CustomerHandler

```
public void addCustomer (int vat, String denomination,  
    int phoneNumber, int discountType)  
    throws ApplicationException {  
    Discount discount = discountCatalog.getDiscount(discountType);  
    customerCatalog.addCustomer(vat, denomination, phoneNumber, discount);  
}
```

SaleSys

Código

CustomerHandler

```
public void addCustomer (int vat, String denomination,  
    int phoneNumber, int discountType)  
    throws ApplicationException {  
    Discount discount = discountCatalog.getDiscount(discountType);  
    customerCatalog.addCustomer(vat, denomination, phoneNumber, discount);  
}
```

DiscountCatalog

```
public Discount getDiscount (int discountType) throws ApplicationException {  
    Discount d = discounts.get(discountType);  
    if (d == null)  
        throw new ApplicationException ("Discount type " + discountType + " not found.");  
    return d;  
}
```

SaleSys

Código

DiscountCatalog

```
public Discount getDiscount (int discountType) throws ApplicationException {  
    Discount d = discounts.get(discountType);  
    if (d == null)  
        throw new ApplicationException ("Discount type " + discountType + " not found.");  
    return d;  
}
```

CustomerHandler

```
public void addCustomer (int vat, String denomination,  
    int phoneNumber, int discountType)  
    throws ApplicationException {  
    Discount discount = discountCatalog.getDiscount(discountType);  
    customerCatalog.addCustomer(vat, denomination, phoneNumber, discount);  
}
```

CustomerCatalog

```
public void addCustomer (int vat, String designation, int phoneNumber, Discount discountType)  
    throws ApplicationException {  
    if (!Customer.isValidVAT(vat))  
        throw new ApplicationException ("Invalid VAT number");  
    if (customers.containsKey(vat))  
        throw new ApplicationException ("Customer already exists");  
    customers.put(vat, new Customer(vat, designation, phoneNumber, discountType));  
}
```

SaleSys

Código

CustomerHandler

```
public void addCustomer (int vat, String denomination,
                        int phoneNumber, int discountType)
    throws ApplicationException {
    Discount discount = discountCatalog.getDiscount(discountType);
    customerCatalog.addCustomer(vat, denomination, phoneNumber, discount);
}
```

DiscountCatalog

```
public Discount getDiscount (int discountType) throws ApplicationException {
    Discount d = discounts.get(discountType);
    if (d == null)
        throw new ApplicationException ("Discount type " + discountType + " not found.");
    return d;
}
```

CustomerCatalog

```
public void addCustomer (int vat, String designation, int phoneNumber, Discount discountType)
    throws ApplicationException {
    if (!Customer.isValidVAT(vat))
        throw new ApplicationException ("Invalid VAT number");
    if (customers.containsKey(vat))
        throw new ApplicationException ("Customer already exists");
    customers.put(vat, new Customer(vat, designation, phoneNumber, discountType));
}
```

Presentation

Business Logic

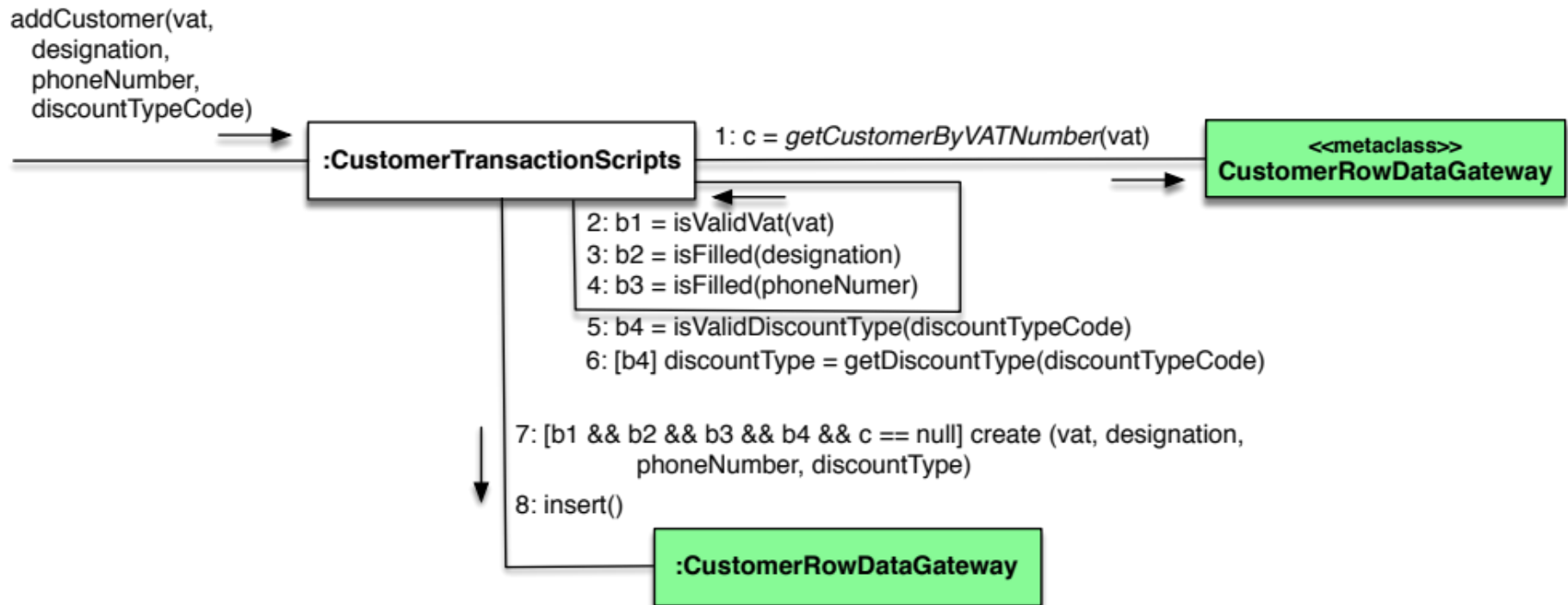
Data Source

Transaction Script

- Organizar o código baseado em casos de uso, de forma procedimental.
- Cada *Script* lida com um caso de uso (recebe input, valida e opera na camada de dados e sistemas externos).
- **Vantagens:** Fácil de entender e de implementar (o que acontece está tudo num sítio), a persistência é fácil de implementar.
- **Desvantagens:** Tendência para código duplicado (não se usa polimorfismo ou herança). Não adequado para quando a lógica é muito complexa.

SaleSys

Adicionar Cliente



Elementos da camada de dados

SaleSys

Código

AddCustomerScript

```
public void addCustomer(int vat, String denomination, int phoneNumber, int discountCode)
    throws ApplicationException {
    // Checks if it is a valid VAT number
    if (!isValidVAT(vat))
        throw new ApplicationException("Invalid VAT number: " + vat);

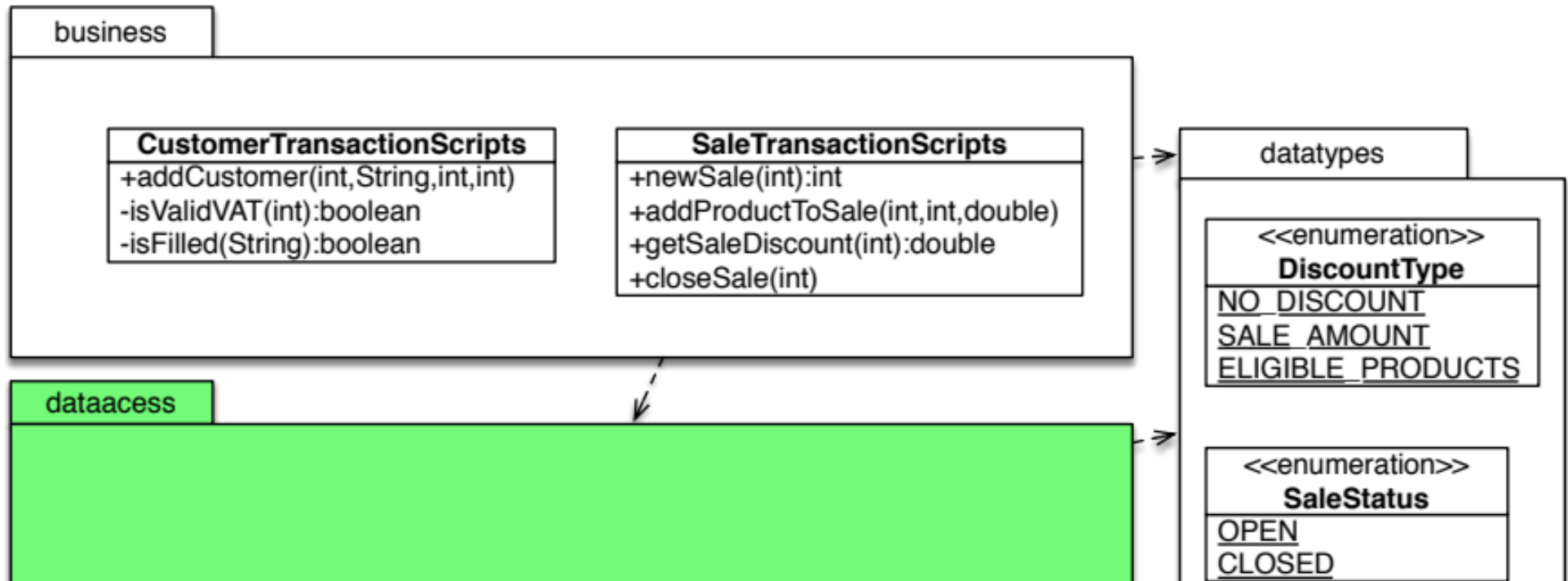
    // Checks if the discount code is valid.
    if (discountCode <= 0 || discountCode > DiscountType.values().length)
        throw new ApplicationException ("Invalid Discount Code: " + discountCode);

    // Checks that denomination and phoneNumber and filled in
    if (!isFilled(denomination) || phoneNumber == 0)
        throw new ApplicationException(
            "Both denomination and phoneNumber must be filled");

    /*
    Tries to stores the customer in the database. There is a
    unique key constraint associated with the VAT number, so if the
    VAT number already exists in the database a persistence exception will
    be thrown.*/
}
```

SaleSys

TransactionScript



Presentation

Business Logic

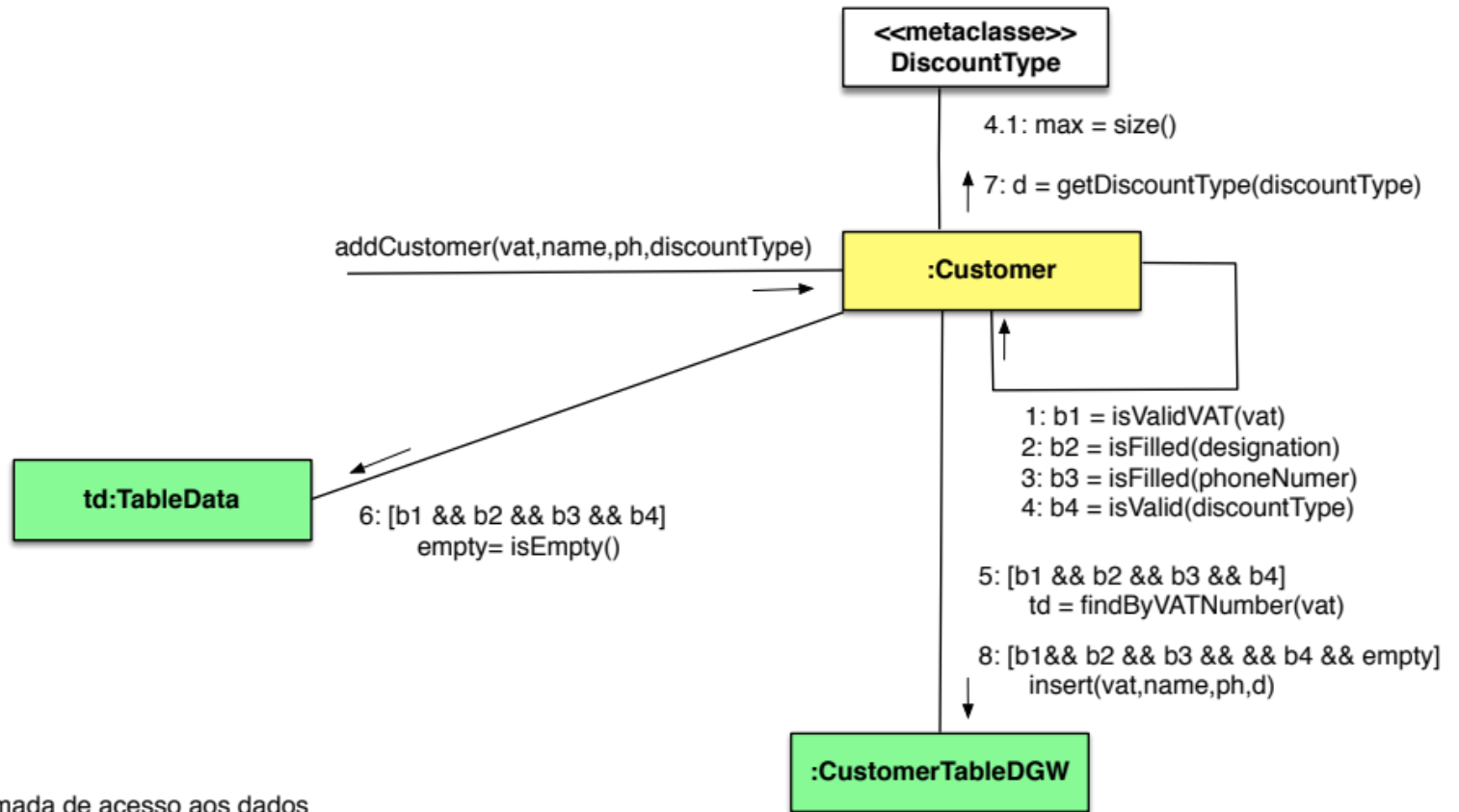
Data Source

Table Module

- Organizar o código baseado em tabelas, e os dados são conjuntos de registos.
- Uma classe por tabela, com métodos que manipulam objectos que representam conjuntos de linhas.
- **Vantagens:** Fácil de persistir, não tem repetição de código (do TS)
- **Desvantagens:** Não é OO, pois os registos não têm identidade, só as tabelas. Também não tira partido de polimorfismo ou herança.

SaleSys

Adicionar Cliente



 Elementos da camada de acesso aos dados

 Table Module

SaleSys

Código

CustomerTableModule

```
public void addCustomer (int vat, String designation, int phoneNumber,
    int discountCode) throws ApplicationException {
    // Checks if it is a valid VAT number
    if (!isValidVAT (vat))
        throw new ApplicationException ("Invalid VAT number: " + vat);

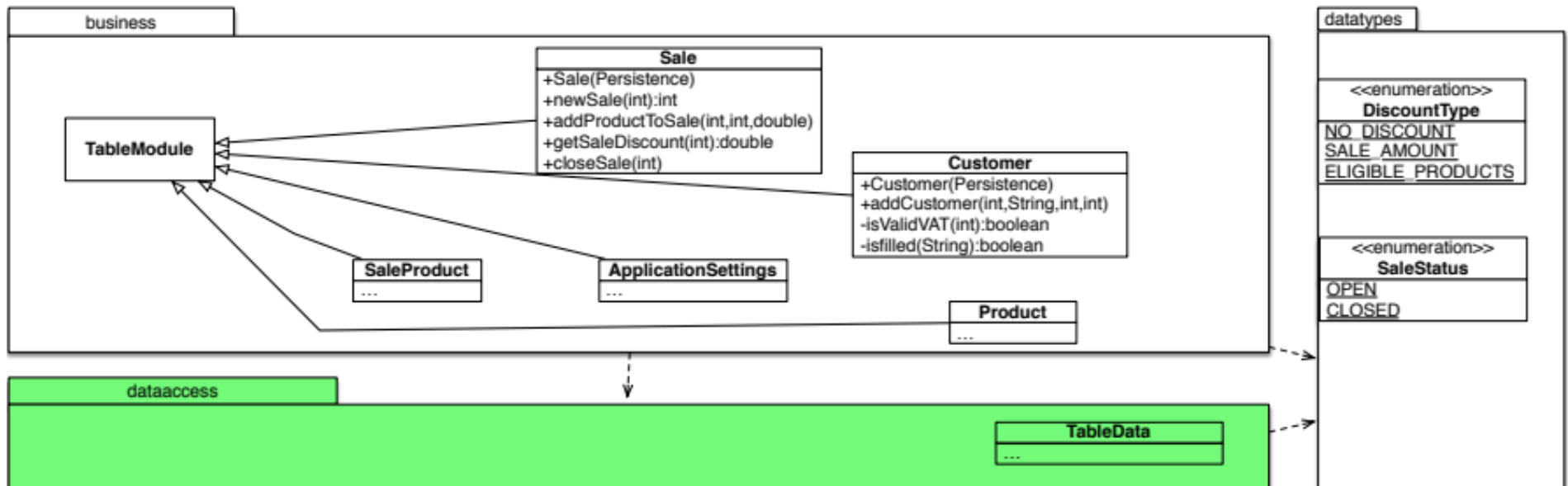
    // Checks if the discount code is valid.
    if (discountCode <= 0 || discountCode > DiscountType.values().length)
        throw new ApplicationException ("Invalid Discount Code: " + discountCode);

    // Checks that there is no other customer with the same VAT number
    if (existsCustomer(vat))
        throw new ApplicationException ("Customer with VAT number " + vat + " already exists!");

    try {
        // If the customer does not exists, add it to the database
        table.insert(vat, designation, phoneNumber, DiscountType.values()[discountCode-1]);
    } catch (PersistenceException e) {
        throw new ApplicationException ("Internal error adding a customer", e);
    }
}
```

SaleSys

Table Module



Presentation

Business Logic

Data Source

Service Layer

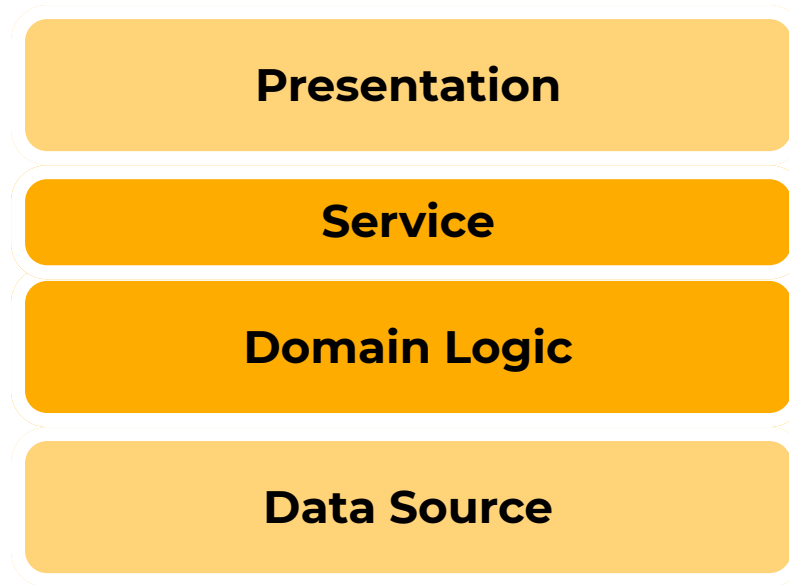
- Fachada que define a API da camada.
- Permite definir endpoints remotos
- Pode ter o controlo transaccional e de segurança.
- A implementação vai para a camada da Lógica do Domínio.

Presentation

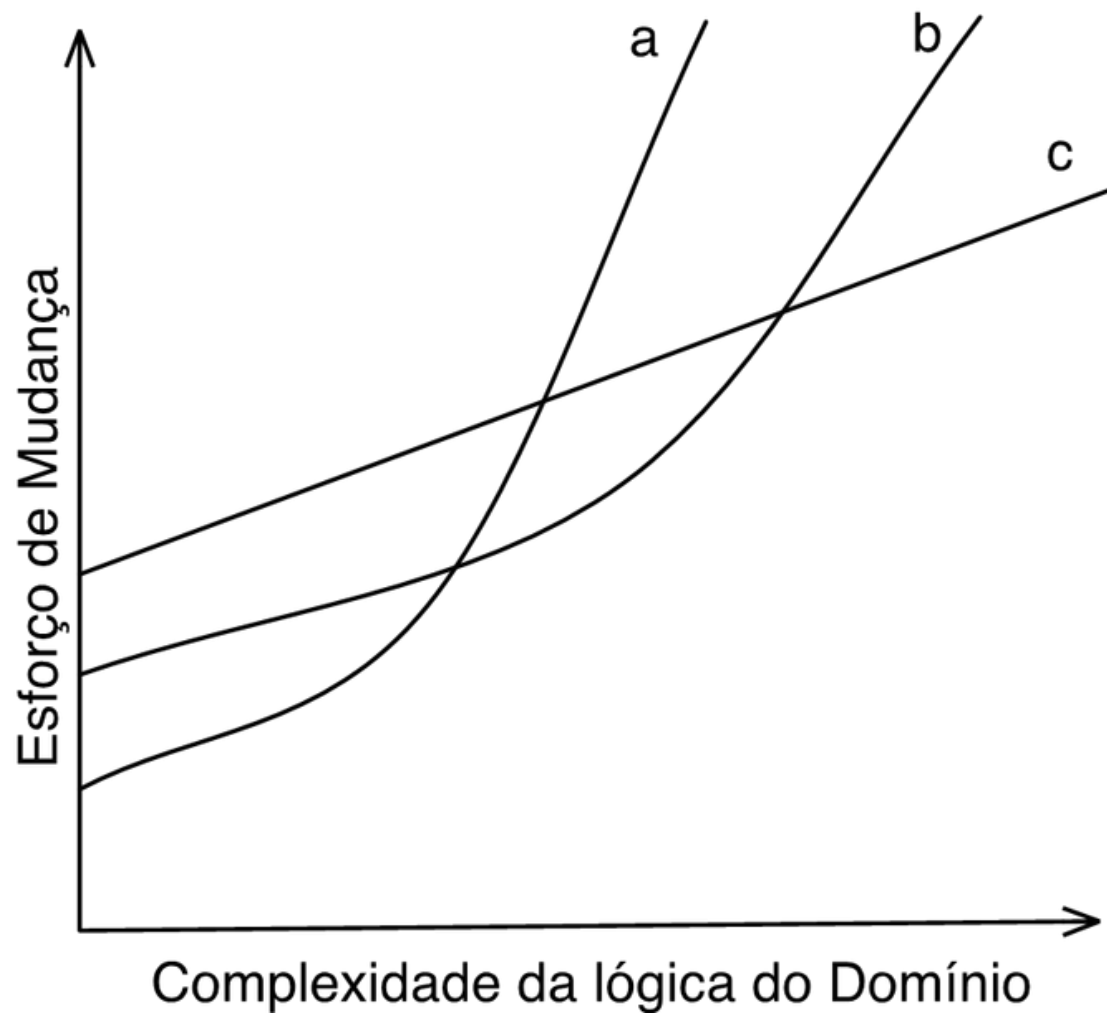
Service

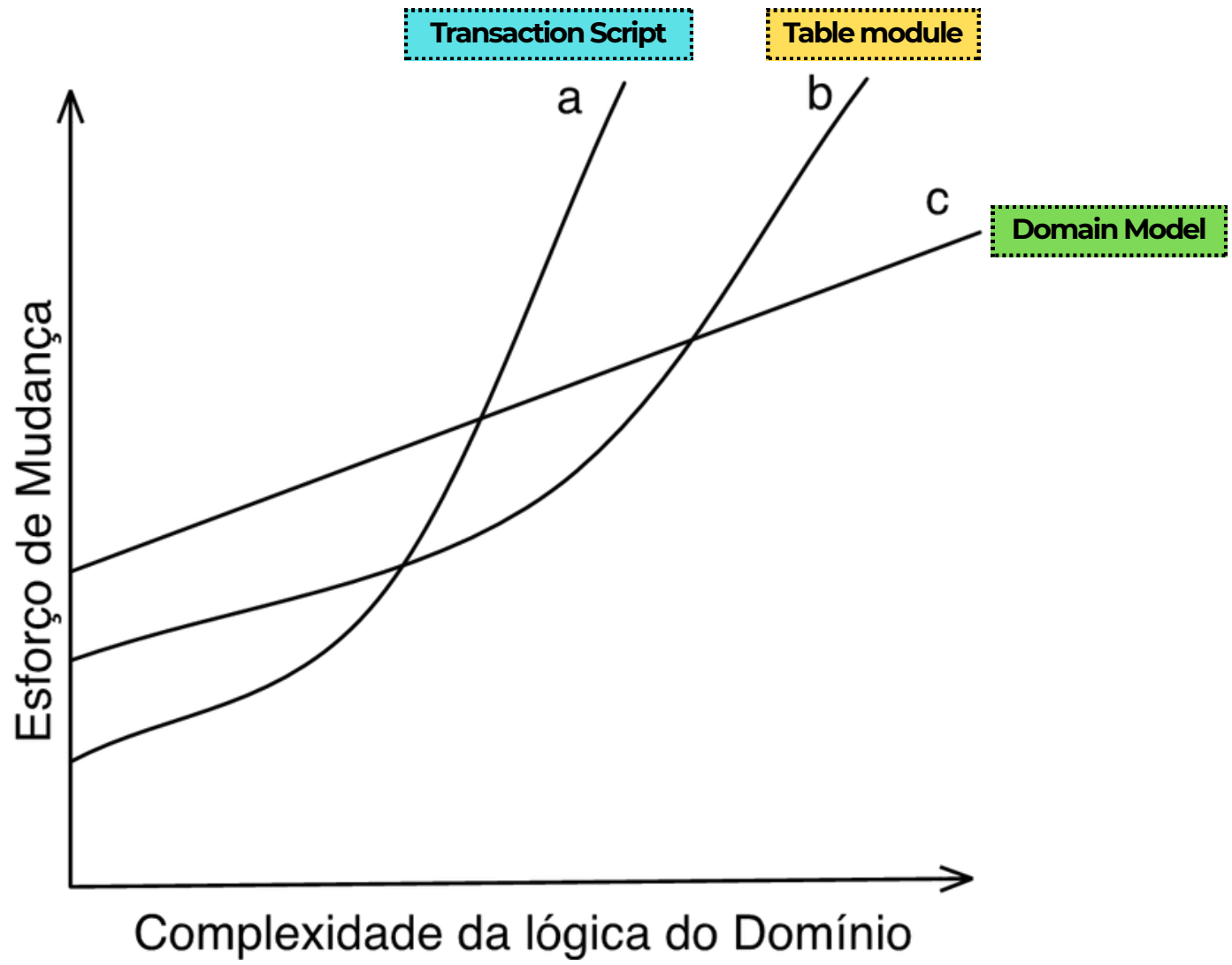
Domain Logic

Data Source



- Service Layer é útil quando:
 - Temos mais do que um tipo de cliente (user-interface)
 - Temos transacções que envolvem recursos diferentes
- Quando a lógica é pouco complexa (e.g., Transaction Script), a própria camada de business pode ser chamada Service Layer e tem classes gordas.





Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model
	Table Module
	Transaction Script
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper

Padrões para as várias camadas

Camada de Apresentação	MVC Front Controller Page Controller Page template
Camada de Negócio	Service Layer
	Domain Model
	Table Module
	Transaction Script
Camada de Dados	Row data gateway
	Table data gateway
	Active Record
	Data Mapper

ORM ?

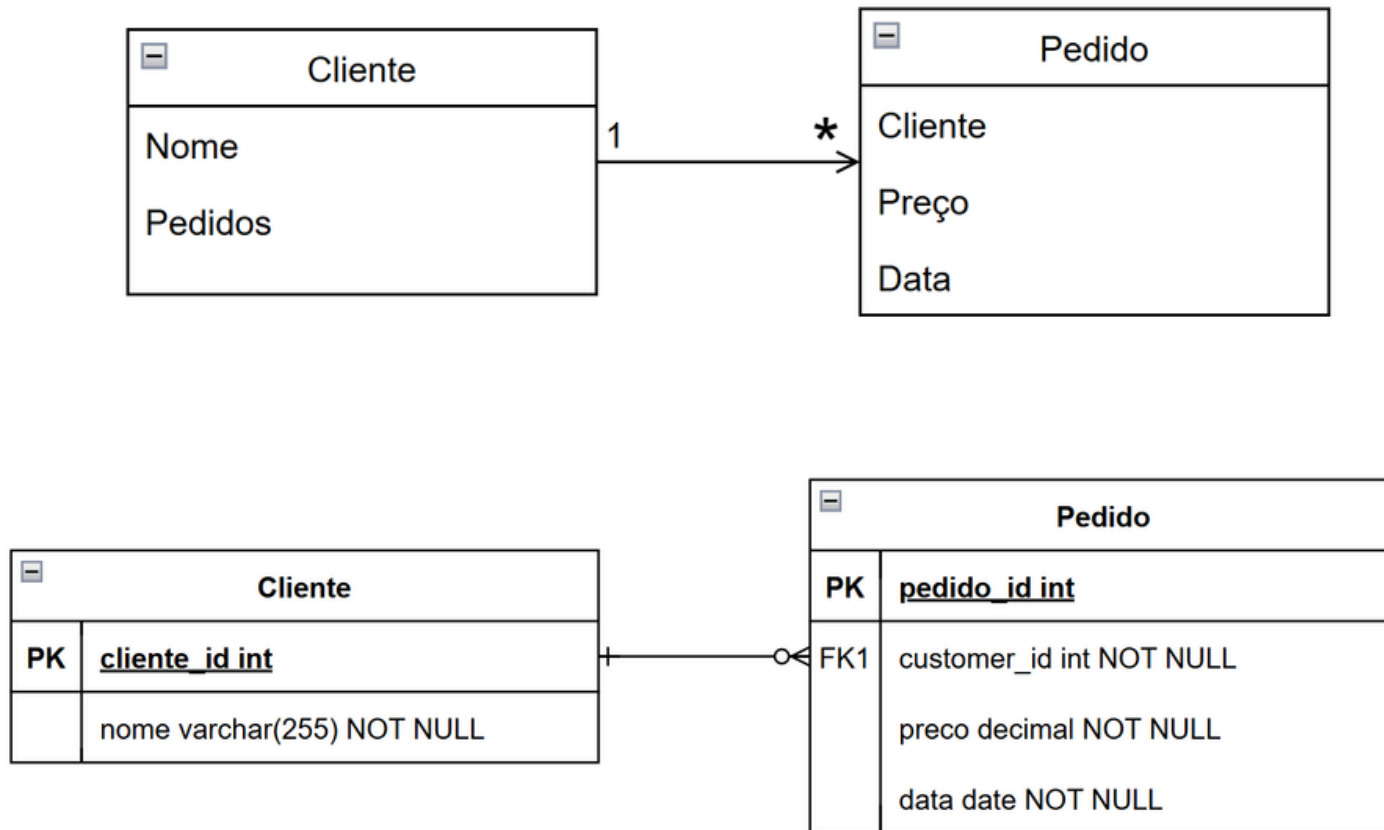
ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

- Referência vs Chave Primária (*)
- Atributos com listas de valores vs foreign key/listas
- Herança vs ???
- Ligações unidirecionais vs relações simétricas
- Navegar referências vs table joins

ORM: Object-Relational Mapping

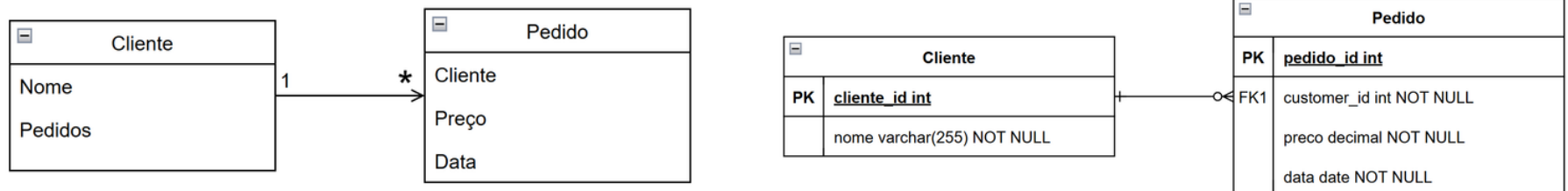
Object-relational Impedance Mismatch



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

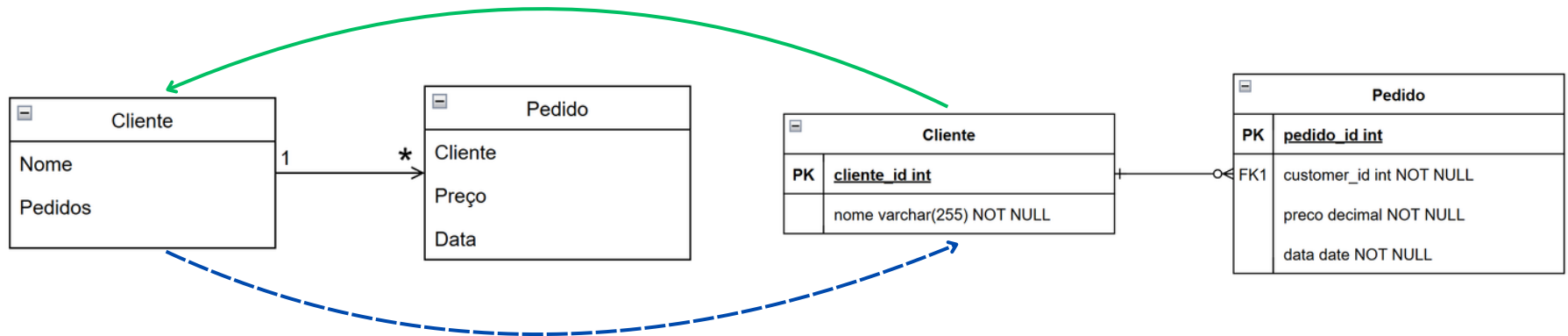
```
Cliente c = clientRepository.findByName("Livia");  
c.setName("Livia Oliveira");  
clientRepository.save(c);
```



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

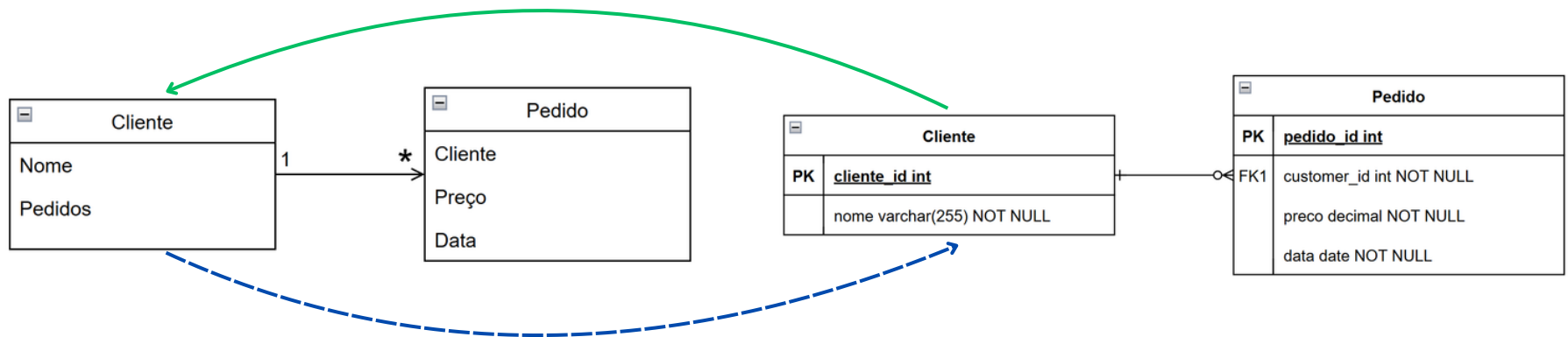
```
Cliente c = clientRepository.findByName("Livia");  
c.setName("Livia Oliveira");  
clientRepository.save(c);
```



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

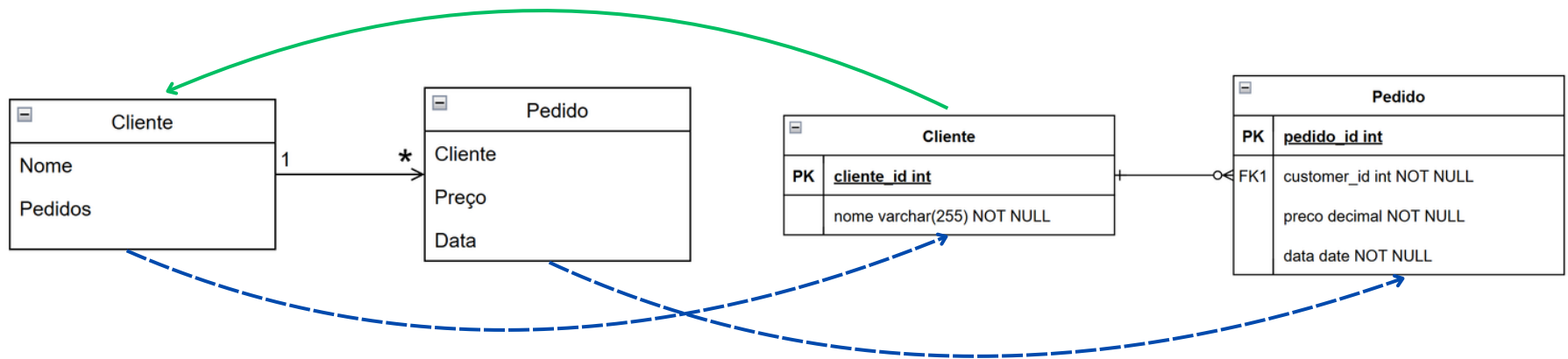
```
Cliente c = clientRepository.findByName("Livia");  
Pedido p = c.getPedidoRecente();  
p.setPreco(100);  
clientRepository.save(c);
```



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

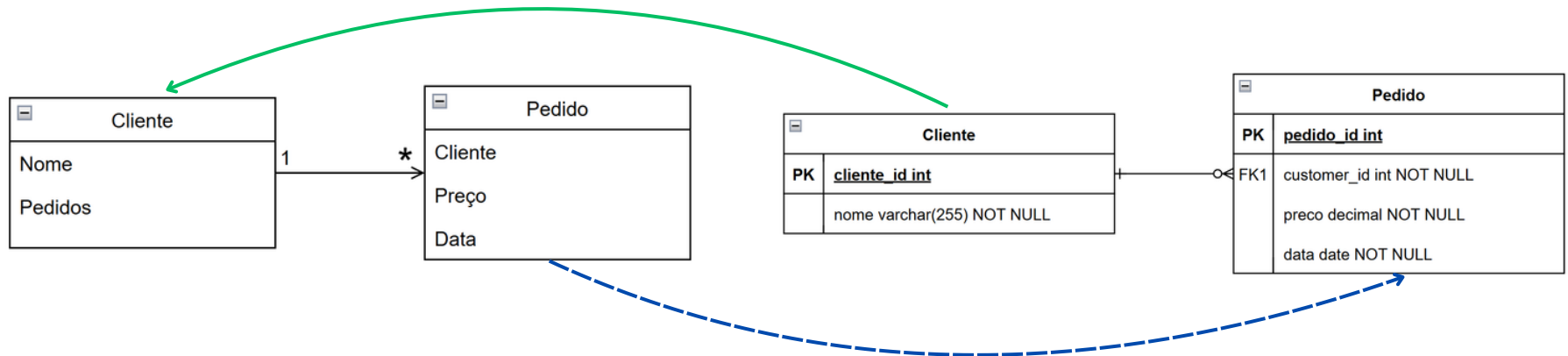
```
Cliente c = clientRepository.findByName("Livia");  
Pedido p = c.getPedidoRecente();  
p.setPreco(100);  
clientRepository.save(c);  
pedidoRepository.save(p);
```



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

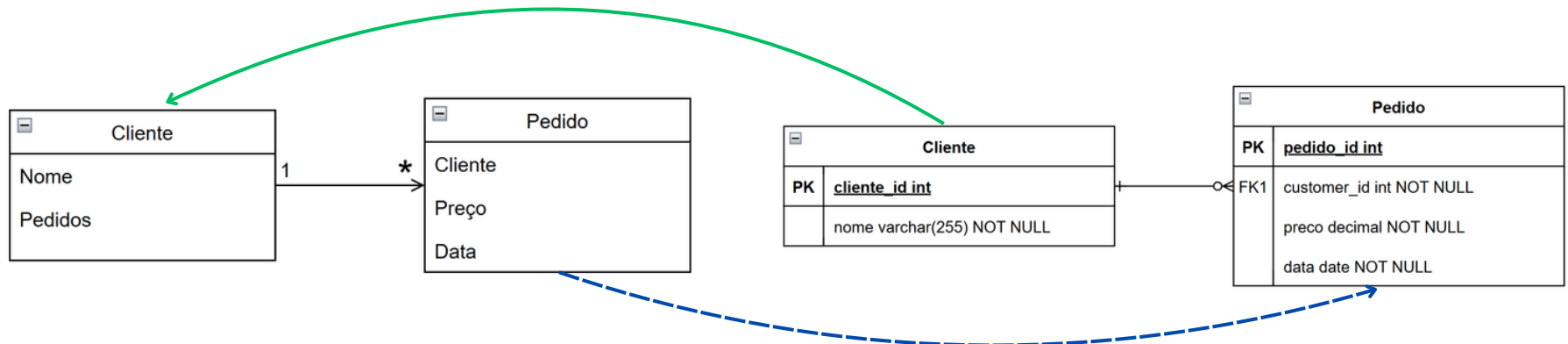
```
Cliente c = clientRepository.findByName("Livia");  
Pedido p = c.getPedidoRecente();  
p.setPreco(100);  
clientRepository.save(c);  
pedidoRepository.save(p);
```



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

```
Cliente c = clientRepository.findByName("Livia");  
Pedido p = new Pedido(c, 10, today);  
pedidoRepository.save(p);
```

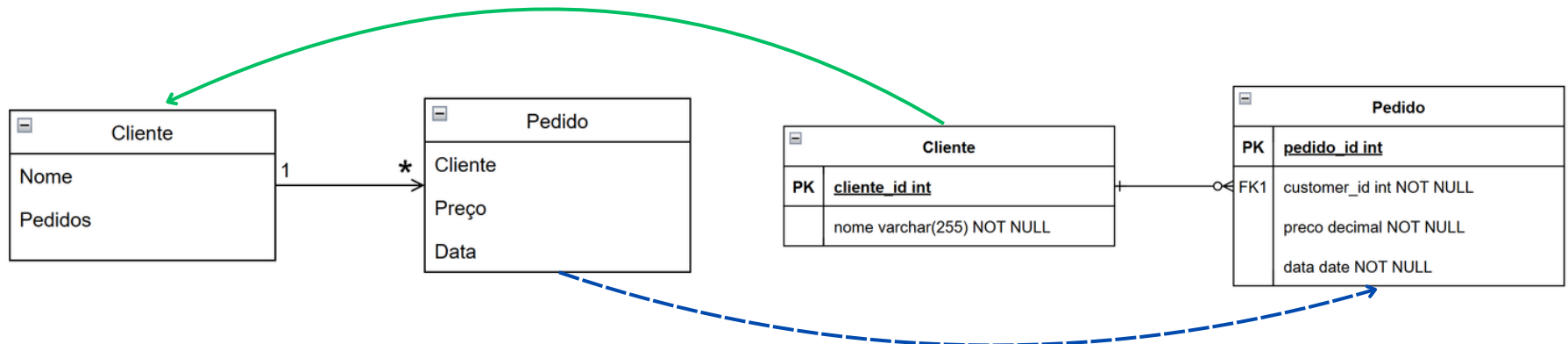


ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

```
Cliente c = clientRepository.findByName("Livia");  
Pedido p = new Pedido(c, 10, today);  
pedidoRepository.save(p);
```

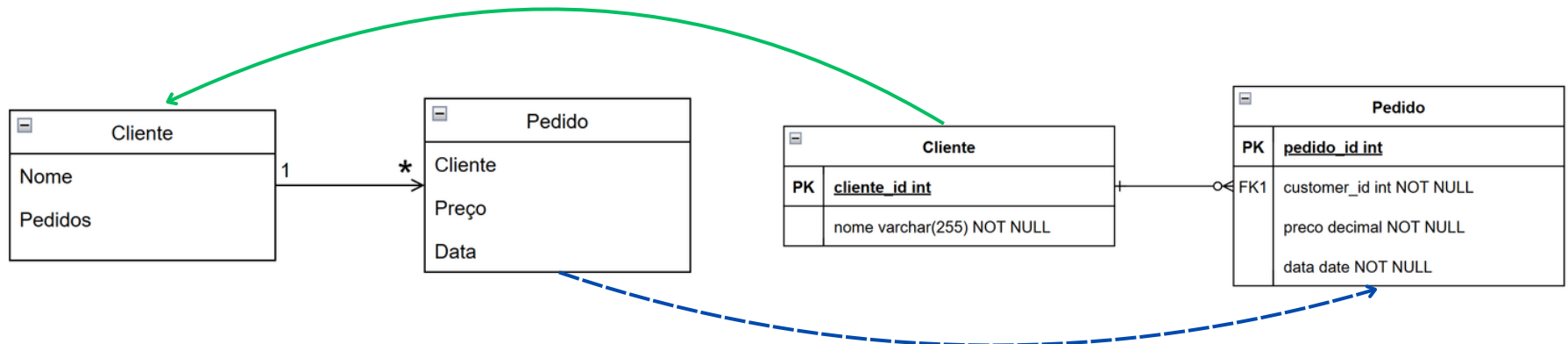
```
System.out.println(c.getPedidos());
```



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

```
Cliente c = clientRepository.findByName("Livia");  
Pedido p = new Pedido(c, 10, today);  
pedidoRepository.save(p);  
  
c.addPedido(p);
```



ORM: Object-Relational Mapping

Object-relational Impedance Mismatch

- Carregar o mesmo registo duas vezes dá uma vista inconsistente
- Lidar com transacções (mudanças concorrentes ao mesmo objecto)
- Acesso a referências:
 - Strict load
 - *pode carregar a base de dados toda para memória, porque tudo pode estar ligado*
 - Lazy load
 - *pode ser lento a aceder campos comuns.*
 - *N + 1 problem*

ORM: Object-Relational Mapping

Padrões

- Estruturais
 - Identity Field
 - Foreign Key Mapping
 - Association Table Mapping
 - Dependent Mapping
 - Embedded Value
 - Single Table Inheritance
 - Class Table Inheritance
 - Concrete Table Inheritance
- Comportamentais
 - Identity Map
 - Unit of Work
 - Lazy Load
 - Optimistic Offline Lock
 - Query Object

ORM: Object-Relational Mapping

JPA



JPA

Provides standards based Object-Relational persistence solution with additional support for many advanced features. EclipseLink JPA provides advanced support for leading relational databases and Java containers.



```
package com.example.accessingdatajpa;

import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;

@Entity
public class Customer {

    @Id
    @GeneratedValue(strategy=GenerationType.AUTO)
    private Long id;
    private String firstName;
    private String lastName;

    protected Customer() {}

    public Customer(String firstName, String lastName) {
        this.firstName = firstName;
        this.lastName = lastName;
    }
}
```